



國立臺北科技大學

資訊工程系碩士班

碩士學位論文

**基於案例式學習與動態計畫修復之 API 任
務大型語言模型代理人自我修正機制研究
A Self-Correction Mechanism for LLM Agents in
API Tasks Using Case-Based Learning and Dynamic
Plan Repair**

研究生：陳安琪

指導教授：劉建宏博士

中華民國一百一十五年七月



國立臺北科技大學

資訊工程系碩士班

碩士學位論文

**基於案例式學習與動態計畫修復之 API 任
務大型語言模型代理人自我修正機制研究**
**A Self-Correction Mechanism for LLM Agents in
API Tasks Using Case-Based Learning and Dynamic
Plan Repair**

研究生：陳安琪

指導教授：劉建宏博士

中華民國一百一十五年七月

「學位論文口試委員會審定書」掃描檔

審定書填寫方式以系所規定為準，但檢附在電子論文內的掃描檔須具備以下條件：

1. 含指導教授、口試委員及系所主管的完整簽名。
2. 口試委員人數正確，碩士口試委員至少 3 人、博士口試委員至少 5 人。
3. 若此頁有論文題目，題目應和書背、封面、書名頁、摘要頁的題目相符。
4. 此頁有無浮水印皆可。

摘要

關鍵詞：大型語言模型代理人、API 任務自動化、案例式學習、動態修復

大型語言模型 (Large Language Model, LLM) 具備自然語言理解、推理與內容生成能力；當其作為代理人 (agent) 的決策核心時，可根據使用者目標進行任務規劃與工具選擇，並透過應用程式介面 (Application Programming Interface, API) 與外部系統互動。然而，在多步驟任務中，任一步驟的 API 選擇、參數綁定、資料處理或相依關係發生錯誤，都可能使後續執行偏離原始目標，因此如何利用既有成功案例輔助規劃，並針對局部失敗進行動態修復，是提升 API 任務代理人可靠度的重要議題。本研究以 API 知識圖譜代理人 (API Knowledge Graph Agent, APIKGAgent) 為基礎，提出經驗引導式局部修復 API 知識圖譜代理人 (Experience-guided Local Repair APIKGAgent, ER-APIKGAgent)，首先從訓練資料中的成功任務萃取可跨任務重複使用的原子化經驗 (atomic experience)，並透過語意檢索提供給初始規劃、重新規劃與修復階段參考；其次設計局部修復流程，根據執行錯誤、相依步驟資料與 API 呼叫證據搜尋可能的修復 API，並以插入、取代或刪除步驟的方式調整局部計畫，當錯誤涉及原始計畫結構時則回退至重新規劃。為評估本研究方法，實驗採用 AppWorld 模擬環境，其涵蓋 9 種應用服務與 457 支 API，可呈現跨應用程式的現實多步驟任務；評估以任務是否完成並使環境產生預期狀態變化為準，並透過整體效能比較、逐步消融實驗與失敗案例分析，觀察 local repair 與 atomic experience 對任務執行的影響。研究結果顯示，在 AppWorld test_normal 資料集上，本研究取得完整任務成功率 X%；此外，局部修復對有明確錯誤訊號的情境有效，但面對較複雜的任務結構或外部狀態限制時，僅靠局部修復與經驗提示仍可能不足以穩定恢復。

ABSTRACT

Keyword: LLM agents, API task automation, case-based learning, dynamic plan repair

Large language models (LLMs) have recently been widely adopted in agent systems, where they can plan tasks, select tools, and invoke APIs according to user goals to complete real-world cross-application tasks. In multi-step API tasks, however, an error in API selection, parameter binding, data processing, or dependency handling at any step may cause the subsequent execution to deviate from the original objective. Traditional approaches usually regenerate the entire plan after an error, which may increase execution cost and disrupt the state and data flow accumulated from completed steps. Therefore, incorporating experience-based guidance and local repair mechanisms into an existing planning framework is an important issue for improving the reliability of API task agents. Based on the API Knowledge Graph Agent (APIKGAgent), this study proposes the Experience-guided Local Repair APIKGAgent (ER-APIKGAgent). The proposed system first extracts reusable atomic experiences from successful tasks in the training data and provides them as references for initial planning, replanning, and repair through semantic retrieval. It then introduces a local repair process that searches for possible repair APIs according to execution errors, dependency information from related steps, and API call evidence, and adjusts the local plan through insertion, replacement, or deletion of steps; when an error affects the structure of the original plan, the system falls back to replanning. Experiments were conducted in the AppWorld simulation environment, which contains 9 application services and 457 APIs and reflects realistic multi-step cross-application tasks. Performance was evaluated based on task completion and the expected state changes in the environment, and the effects of local repair and atomic experience were examined through overall performance comparison, stepwise ablation studies, and failure-case analysis. On the AppWorld test_normal dataset we observe an overall task success rate of X%; local repair helps when errors have explicit signals but may be insufficient for failures caused by complex task structures or external state constraints.

誌謝

回顧這段碩士求學的歷程，從最初的摸索到最終完成論文，其中經歷了無數嘗試與修正。能夠走到這一步，並非一人之力，而是來自許多人的陪伴與支持。

首先，誠摯感謝指導教授劉建宏老師。在研究過程中，老師總能在關鍵時刻點出問題核心，引導我重新思考方向。無論是研究上的討論，或是面對困難時的提醒，都讓我逐漸建立起更成熟的思考方式，也學會如何更嚴謹地面對學術問題。同樣感謝梁文耀教授的指導與建議。您在討論中提出的觀點，往往讓我跳脫原有的框架，使研究內容得以更加周全，也讓我對自己的研究有更深一層的理解。

在實驗室的日子裡，感謝珮倫、郁喬、冠穎、以謙等夥伴的陪伴。無論是討論問題時的腦力激盪，或是日常相處中的交流與玩笑，都讓這段研究生活不再只是壓力與進度，而多了許多值得回味的片段。此外，也想感謝一路上關心與支持我的朋友們，在我感到疲憊或迷惘時給予鼓勵，讓我能重新調整步伐，繼續前進。

最後，將這份成果獻給我的家人。謝謝你們長久以來的理解與支持，讓我能專心投入學業，在面對挑戰時依然保有前進的力量。這段旅程的結束，同時也是另一段路的開始。謹以此文，致上我最真摯的感謝。

目錄

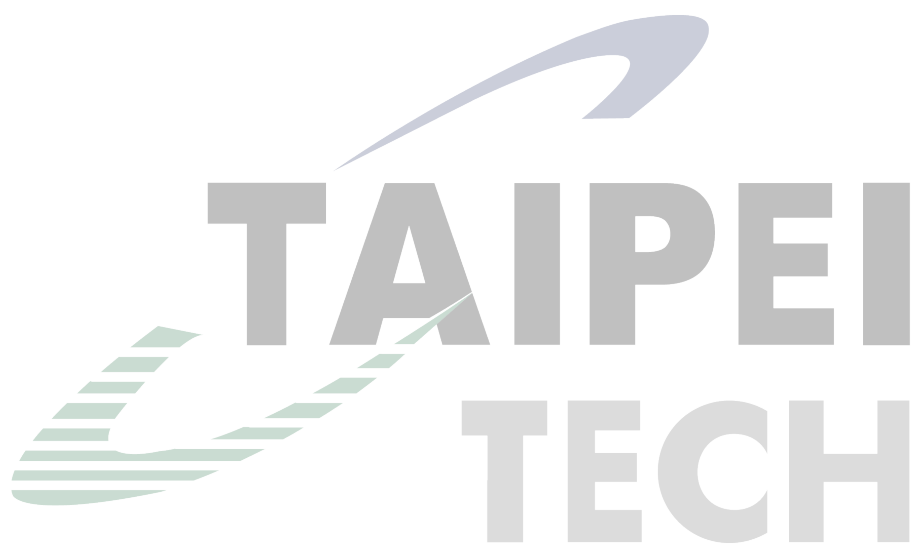
摘要	i
ABSTRACT	ii
誌謝	iii
目錄	iv
圖目錄	vii
表目錄	viii
第一章 緒論	1
1.1 研究動機	1
1.2 研究目的	2
1.3 論文組織架構	3
第二章 相關技術與文獻探討	4
2.1 大型語言模型代理人	4
2.2 API 任務自動化	4
2.3 案例式學習	4
2.4 動態計畫修復	4
第三章 系統方法綜述與架構	5
3.1 基礎架構：APIKGAgent 簡述	5
3.2 本研究問題定義與改良目標	5
3.3 本研究改良架構總覽	5
3.4 成功經驗參考機制	5
3.4.1 經驗來源與整理方式	5
3.4.2 語意檢索與注入流程	5
3.5 動態修復與局部計畫修正機制	5
3.5.1 錯誤偵測與修復策略	5
3.5.2 局部計畫修正流程	5
3.6 經驗累積閉環	5
第四章 系統實作與細節	6

4.1	開發環境與工具	6
4.2	原 APIKGAgent 基礎執行流程與本研究修改點	7
4.2.1	Main Graph / Executor Graph / API Execute Graph 概述	7
4.2.2	Process Step / API Step 概述	7
4.2.3	本研究延伸位置說明	7
4.3	全域成功經驗建立與注入	7
4.3.1	train dataset 成功經驗萃取	7
4.3.2	RAG 語意檢索與 Planner 注入	7
4.4	Rerollout 動態修復實作	7
4.4.1	錯誤分類與修復 API 搜尋	7
4.4.2	任務步驟的新增、刪除與更新	7
4.4.3	與原 replan 機制之差異	7
4.5	區域成功修復經驗儲存與檢索	7
4.5.1	成功修復經驗萃取	7
4.5.2	Local recovery lessons 儲存與後續任務再利用	7
4.6	其他實作細節	7
第五章	實驗與結果分析	8
5.1	研究問題	8
5.2	實驗設置	8
5.2.1	實驗基準	8
5.2.2	評估指標	8
5.2.3	實驗環境	8
5.3	實驗一：整體效能比較	8
5.4	實驗二：逐步式消融實驗	8
5.5	實驗三：動態修復行為與失敗案例分析	8
第六章	結論與未來研究	9
6.1	結論	9
6.2	未來研究	9
	參考文獻	10

附錄	11
6.3 Prompts	11
6.4 程式碼	11



圖目錄



表目錄



第一章 緒論

1.1 研究動機

近年來，大型語言模型（Large Language Model, LLM）已逐漸由傳統的文字生成與問答系統，發展為能夠自主規劃、呼叫工具並操作外部系統的智慧代理人（LLM-based Agent）[1]。透過應用程式介面（Application Programming Interface, API），代理人得以執行資料查詢、訊息傳送、檔案管理、行事曆管理及交易處理等實際工作，使大型語言模型由單純提供資訊的輔助工具，進一步成為能與數位環境互動並完成使用者目標的自動化系統。因此，如何提升代理人在真實環境中的任務完成能力與執行可靠性，已成為大型語言模型研究的重要方向之一。

然而，真實世界中的 API 任務通常涉及多個相互依賴的執行步驟，而非單次 API 呼叫即可完成。代理人除了需要理解使用者需求並選擇適當的 API 外，還必須進行多步驟規劃、維護步驟間的資料依賴關係、保存中間執行結果，並持續遵循任務條件與限制。當任務規模增加或操作流程變得複雜時，任何一個步驟的錯誤都可能影響後續決策，使錯誤沿著執行流程持續傳播，最終導致整體任務失敗。因此，如何降低錯誤傳播對任務執行結果的影響，已成為提升多步驟 API 代理人可靠性的重要挑戰。

本研究延續 APIKGAgent [2] 之研究架構進行探討。APIKGAgent 透過 API 知識圖譜、語意檢索及任務規劃機制協助代理人完成多步驟 API 任務，並具備基本的重新規劃能力。然而，在複雜任務執行過程中仍可觀察到錯誤傳播與局部失敗難以修復等現象。此外，既有架構尚未系統化利用由訓練資料成功任務軌跡所萃取的策略經驗。此現象顯示，除了提升即時規劃能力之外，如何利用由訓練資料成功任務軌跡所萃取的經驗知識輔助決策，以及如何針對局部錯誤進行修復，亦可能是影響整體任務完成能力的重要因素。

綜合上述觀察，目前多步驟 API 任務代理人仍面臨兩項關鍵挑戰：

- **全域經驗利用不足：**既有系統缺乏對訓練資料中成功任務經驗的系統化利用。代理人在不同任務中經常面臨相似的規劃與資料依賴問題，但既有系統尚未將成功任務軌跡所包含的策略知識整理為可檢索的經驗，使代理人在面對相似任務結構時，缺乏可供規劃與修復參考的依據。
- **局部錯誤修復能力不足：**當任務執行過程中發生錯誤時，現有系統多仰賴重新產生整體計畫進行修正。然而，許多錯誤並非來自整體計畫設計不良，而可能僅發生於部分步驟。若直接重新規劃整體任務，不僅增加額外成本，也可能破壞原本正確的執行流程，甚至導致新的錯誤產生。

基於上述觀察，本研究認為提升多步驟 API 代理人可靠性的關鍵，不僅在於大型語言模型本身的推理能力，也在於如何利用由訓練資料成功任務軌跡所萃取的策略知識，並針對執行期間的局部錯誤進行適當修復。因此，本研究提出經驗引導式局部修

復 API 知識圖譜代理人 (Experience-guided Local Repair APIKGAgent, ER-APIKGAgent)，探討如何從訓練資料中的成功任務軌跡萃取細粒度經驗，建立離線全域經驗庫，並透過語意檢索將相關經驗應用於初始規劃、重新規劃與局部修復流程；同時研究如何根據執行結果、步驟相依資料與錯誤證據進行動態局部修復。藉由經驗重用與動態修復機制的結合，本研究將探討全域經驗是否能為相似任務結構提供有效的規劃與修復參考，以及局部修復是否有助於代理人由執行失敗中恢復，進而評估兩項機制對多步驟 API 任務完成能力與錯誤修復表現的影響。

1.2 研究目的

本研究針對大型語言模型代理人在多步驟 API 任務中所面臨之錯誤傳播、訓練資料中成功任務策略尚未被系統化利用，以及局部失敗不易修復等問題，提出經驗引導式局部修復 API 知識圖譜代理人 (ER-APIKGAgent)。本研究延續 APIKGAgent [2] 之架構進行探討，並在其 API 知識圖譜、API 檢索、任務規劃與步驟式執行流程上，加入經驗重用與動態局部修復機制，以提升代理人在複雜 API 任務中的自我修正能力。

具體而言，本研究之目的如下：

1. 建立訓練資料成功經驗的萃取與重用機制

本研究從訓練資料中的成功任務軌跡萃取可跨任務重用的原子化經驗 (Atomic Experience)，將適用情境、規劃原則、必要中間概念與常見陷阱轉換為結構化經驗知識。透過語意檢索機制，系統可在初始規劃、重新規劃與局部修復階段取得與當前情境相關的經驗，以提供代理人規劃與修復時的策略參考，並探討此機制是否有助於減少已知失敗模式在相似任務中出現。

2. 建立動態局部修復機制以處理執行失敗

本研究探討代理人在執行失敗時，如何根據失敗步驟、錯誤訊息、相依資料與 API 執行結果判斷修復方向。當錯誤可由局部調整解決時，系統透過插入、替換或刪除步驟修正局部計畫；當錯誤涉及整體任務理解或計畫結構時，則回退至重新規劃，以減少不必要的整體重新規劃，並降低重新規劃對既有正確步驟與資料流程造成干擾的可能性。

3. 建構具備自我修正能力之多步驟 API 任務代理人

本研究整合經驗重用與動態局部修復機制，建構 ER-APIKGAgent，使代理人能於規劃、執行、錯誤偵測、經驗檢索、局部修復與重新規劃之間形成完整的自我修正流程。藉由此架構，本研究期望提升代理人在多步驟 API 任務中的任務完成能力與失敗恢復能力。

4. 探討經驗重用與動態修復機制之影響

本研究進一步探討經驗重用與動態修復機制對代理人自我修正能力之影響，並分析其於不同錯誤情境下的適用性與限制。透過 AppWorld [3] 多應用程式 API 任務環境進行評估，本研究期望釐清上述機制對任務完成率與錯誤修復表現的實際影響。

綜合而言，本研究旨在驗證：結合由訓練資料成功任務軌跡所萃取之全域經驗與動態局部計畫修復，是否能強化大型語言模型代理人在多步驟 API 任務中的自我修正能力，並進一步釐清兩項機制於不同錯誤情境下的效益與能力邊界。

1.3 論文組織架構

本論文共分為六章，各章內容概述如下：

- **第一章緒論：**說明研究背景、研究動機與研究目的，並概述本論文的整體章節安排。
- **第二章相關技術與文獻探討：**整理大型語言模型代理人、API 任務自動化、案例式學習及動態計畫修復等相關技術與研究。
- **第三章系統方法綜述與架構：**簡述 APIKGAgent 基礎架構，定義本研究欲改善的問題，並介紹全域經驗萃取與檢索、局部動態修復及整體自我修正流程等方法設計。
- **第四章系統實作與細節：**說明開發環境、原 APIKGAgent 執行流程與修改位置，以及原子化經驗建立與注入、rerollout 動態修復、錯誤證據整理及實驗控制等實作細節。
- **第五章實驗與結果分析：**說明研究問題、實驗設置與評估指標，並透過整體效能比較、逐步式消融實驗及失敗案例分析評估本研究方法。
- **第六章結論與未來研究：**總結研究成果與主要貢獻，說明目前方法的限制，並提出後續可改進與延伸的研究方向。

第二章 相關技術與文獻探討

2.1 大型語言模型代理人

本節整理大型語言模型作為代理人核心時的基本概念、能力邊界與常見設計方式。

2.2 API 任務自動化

本節整理 API 任務規劃、參數綁定、工具呼叫與多步驟執行流程的相關研究。

2.3 案例式學習

本節說明案例式學習如何透過既有成功案例支援推論、規劃與錯誤修正。

2.4 動態計畫修復

本節整理任務執行過程中，如何根據錯誤訊號與局部狀態修正原始計畫的相關方法。

第三章 系統方法綜述與架構

3.1 基礎架構：APIKAgent 簡述

在此說明 APIKAgent 的基本設計、資料流與任務執行流程。

3.2 本研究問題定義與改良目標

在此定義本研究欲解決的問題，以及相對於原系統的改良目標。

3.3 本研究改良架構總覽

在此總覽本研究提出的整體架構與模組關係。

3.4 成功經驗參考機制

3.4.1 經驗來源與整理方式

3.4.2 語意檢索與注入流程

3.5 動態修復與局部計畫修正機制

3.5.1 錯誤偵測與修復策略

3.5.2 局部計畫修正流程

3.6 經驗累積閉環

在此說明成功經驗如何回饋到後續任務中形成閉環。

第四章 系統實作與細節

4.1 開發環境與工具

本研究採用 AppWorld [3] 作為多應用程式 API 任務的執行與評估環境。本節將說明系統開發所使用的程式語言、框架、資料庫、模型服務及相關工具。



4.2 原 APIKGAgent 基礎執行流程與本研究修改點

4.2.1 Main Graph / Executor Graph / API Execute Graph

概述

4.2.2 Process Step / API Step 概述

4.2.3 本研究延伸位置說明

4.3 全域成功經驗建立與注入

4.3.1 train dataset 成功經驗萃取

4.3.2 RAG 語意檢索與 Planner 注入

4.4 Rerollout 動態修復實作

4.4.1 錯誤分類與修復 API 搜尋

4.4.2 任務步驟的新增、刪除與更新

4.4.3 與原 replan 機制之差異

4.5 區域成功修復經驗儲存與檢索

4.5.1 成功修復經驗萃取

4.5.2 Local recovery lessons 儲存與後續任務再利用

4.6 其他實作細節

第五章 實驗與結果分析

5.1 研究問題

在此列出本章實驗欲驗證的研究問題。

5.2 實驗設置

5.2.1 實驗基準

5.2.2 評估指標

5.2.3 實驗環境

本研究於 AppWorld [3] 環境中進行實驗，並使用其任務資料與狀態式評估機制驗證代理人的任務完成情形。

5.3 實驗一：整體效能比較

5.4 實驗二：逐步式消融實驗

5.5 實驗三：動態修復行為與失敗案例分析

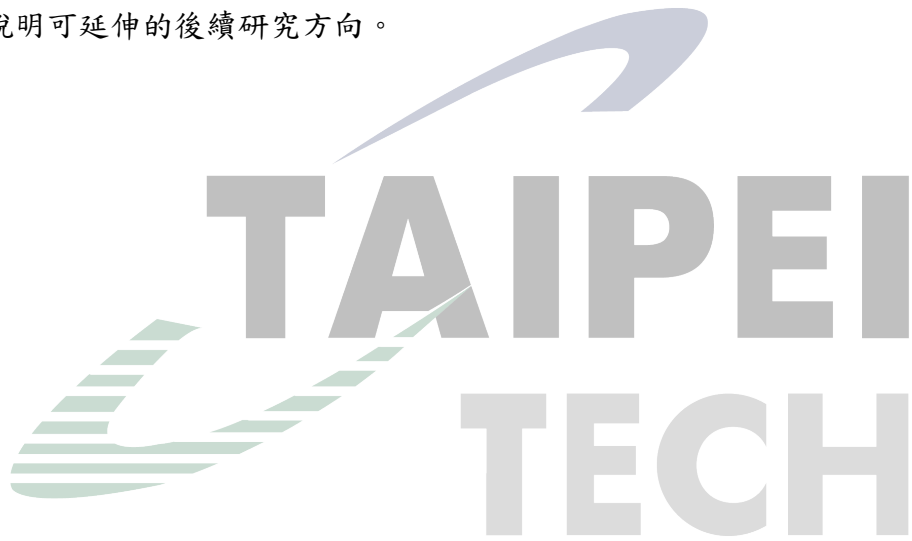
第六章 結論與未來研究

6.1 結論

在此總結本研究的主要貢獻與實驗結果。

6.2 未來研究

在此說明可延伸的後續研究方向。



參考文獻

- [1] L. Wang, C. Ma, X. Feng, *et al.*, “A survey on large language model based autonomous agents,” *Front. Comput. Sci.*, vol. 18, no. 6, 186345, Mar. 2024. DOI: 10.1007/s11704-024-40231-1.
- [2] 吳柏諭，一個由 LLM 驅動之 AI 代理人透過操作 API 自動完成任務之研究，碩士論文，國立臺北科技大學資訊工程系，2025。 <https://hdl.handle.net/11296/837n46>.
- [3] H. Trivedi, T. Khot, M. Hartmann, *et al.*, “AppWorld: A controllable world of apps and people for benchmarking interactive coding agents,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 16 022–16 076. DOI: 10.18653/v1/2024.acl-long.850.



附錄

6.3 Prompts

在此整理撰寫論文、設計實驗與系統實作時使用的 prompts。

6.4 程式碼

在此整理重要程式碼片段與關鍵實作說明。

